

Оглавление

1. Постановка задачи
 2. Заданный вариант
 3. Алгоритмы работы программного комплекса ВКО
 - 3.1. Общий блок инициализации и управления процессами
 - 3.2. Подсистема радиолокационной разведки (Алгоритм работы РЛС)
 - 3.3. Подсистема огневого поражения (Алгоритмы ЗРДН и СПРО)
 - 3.4. Подсистема криптографической защиты данных
 - 3.5. Информационно-аналитический центр (Алгоритм Командного пункта)
 4. Топографическая карта с нанесенными зонами и параметрами работы комплексов ВКО
 5. Отрывки журналов работы (SQL-запросы к БД и вывод ответов)
- Вывод

1. Постановка задачи

Цель работы: разработать и исследовать программную модель автоматизированной системы воздушно-космической обороны (ВКО) с использованием Bash. Модель должна представлять собой распределенную систему, способную в режиме реального времени обрабатывать данные о воздушной обстановке, принимать решения на перехват и вести защищенный учет боевых действий. Для достижения поставленной цели необходимо решить следующие основные задачи:

Реализация асинхронной архитектуры: разработать набор независимых скриптов (РЛС, ЗРДН, СПРО), функционирующих в фоновом режиме. Обеспечить корректный запуск, мониторинг состояния (PID) и безопасное завершение процессов без утечек ресурсов ОС.

Организация межпроцессного взаимодействия: настроить непрерывное чтение данных о воздушных целях из файловой системы.

Разработка алгоритмов селекции и сопровождения целей: реализовать математический аппарат для фильтрации целей по дальности, попаданию в

сектор обзора и скорости. Обеспечить классификацию целей на аэродинамические и баллистические.

Проектирование логики огневого поражения: внедрить строгий учет боекомплекта каждой огневой установки.

Обеспечение информационной безопасности: разработать собственный алгоритм криптографического преобразования логов боевых систем на основе поточного кодирования Base64 и зеркального реверса строк для защиты данных от визуального прочтения и несанкционированного доступа.

Создание Командного пункта: реализовать модуль автоматической дешифровки логов, хронологической сортировки и импорта данных в реляционную базу данных SQLite3. Настроить SQL-запросы для вывода аналитики.

2. Заданный вариант

Условие загружено по ссылке: <https://ibb.co/Rk555YqL>

3. Алгоритмы работы программного комплекса ВКО

3.1. Общий блок инициализации и управления процессами

Каждый боевой скрипт при запуске выполняет один и тот же базовый алгоритм подготовки:

Проверка окружения: скрипт проверяет, что он запущен в Linux (Bash) и не от имени root.

Чтение конфигурации: выполняется поиск файла config.cfg в текущей папке. Если файл найден, скрипт подгружает из него свои параметры (Имя, Координаты X и Y, Радиус действия, Боекомплект, углы обзора).

Защита от двойного запуска: скрипт проверяет наличие своего pid файла. Если файл есть и процесс с таким номером еще жив в системе, запуск отменяется. Иначе скрипт записывает свой текущий номер процесса (PID) в файл и вешает системный перехватчик, чтобы при закрытии скрипта удалить этот файл.

Подготовка директорий: если папки /tmp/GenTargets/Destroy нет, скрипт её создает.

Старт цикла: скрипт рапортует в терминал: "Запущен. Ждем цели..." и уходит в бесконечный цикл while true с шагом в 1 секунду.

3.2. Подсистема радиолокационной разведки (Алгоритм работы РЛС)

Находясь в главном цикле, радар делает следующее:

Сканирование: читает до 50 самых свежих файлов из папки /tmp/GenTargets/Targets. Если генератор успел удалить файл во время чтения, ошибка подавляется (2>/dev/null).

Дешифровка ID: имя файла расшифровывается утилитой xxd в читаемый ID цели.

Снятие координат: скрипт читает первую строчку файла цели и извлекает оттуда текущие координаты X и Y.

Проверка видимости: рассчитывается дистанция от РЛС до цели: $D = \sqrt{\Delta X^2 + \Delta Y^2}$. Если цель дальше радиуса — она игнорируется.

Рассчитывается азимут цели относительно РЛС. Если цель не попадает в сектор обзора — она игнорируется.

Вычисление скорости: если цель видима, скрипт ждет следующей секунды, чтобы получить её новые координаты. Затем вычисляет скорость.

Фильтрация: РЛС настроена только на баллистические цели. Если $V \geq 8000$, РЛС вызывает функцию `write_log` и сообщает об обнаружении.

Анализ вектора: РЛС рассчитывает расстояние от цели до координат базы СПРО в прошлую секунду D_1 и в текущую D_2 . Если $D_2 < D_1$, значит цель летит прямо на базу ПРО. Записывается лог: "Цель движется в направлении СПРО".

Шифрование лога: функция `write_log` кодирует сообщение в Base64, переворачивает строку задом наперед (`rev`) и пишет в индивидуальный файл `*_enc.log`.

3.3. Подсистема огневого поражения (Алгоритмы ЗРДН и СПРО)

Боевые установки работают сложнее, так как они управляют ракетами и оценивают результат стрельбы:

Чтение эфира и координат: (аналогично РЛС: сканирование папки, дешифровка ID, извлечение X и Y, проверка вхождения в радиус поражения).

Проверка исчезнувших целей: скрипт проверяет внутренний массив `FIRE_D_TARGETS` (очередь целей, по которым был произведен пуск). Если цели из очереди больше нет в папке генератора, фиксируется: "ПОДТВЕРЖДЕНО: Цель пропала с радаров".

Проверка остановки: скрипт высчитывает разницу координат цели за 1 секунду: ΔX и ΔY . Если по цели был произведен пуск, и её $\Delta X = 0$ и $\Delta Y = 0$, скрипт фиксирует попадание: "УНИЧТОЖЕНА!". Цель удаляется из очереди обстрела. Если цель всё еще движется, фиксируется "ПРОМАХ!", и цель возвращается в список доступных для атаки.

Игнорирование обломков: если $\Delta X = 0$ и $\Delta Y = 0$, но мы по ней сейчас не стреляли (это "мертвая" цель, сбита кем-то другим), скрипт её просто пропускает, чтобы не тратить ракеты.

Классификация: вычисляется скорость V . Для ЗРДН: берем цели ≤ 1000 (КР или Самолет). Для СПРО: берем цели ≥ 8000 (БР).

Алгоритм стрельбы:

Если боекомплект ≤ 0 : пуск блокируется, в лог пишется "БОЕПРИПАСЫ ОТСУТСТВУЮТ".

Если боекомплект > 0 : пишется лог "Обнаружена цель". Происходит "выстрел": скрипт создает в папке Destroy файл с именем ID цели, внутри которого пишет свое имя (например, ZRDN_1). Боекомплект уменьшается на 1. ID цели заносится в массив FIRED_TARGETS, чтобы на следующей секунде проверить попадание (шаг 3). Пишется лог "Произведен пуск. Остаток...".

Шифрование лога: все сообщения прогоняются через Base64 и rev, после чего сохраняются в файл *_enc.log.

3.4. Подсистема криптографической защиты данных

Для имитации защиты каналов передачи данных от несанкционированного доступа (НСД) и предотвращения визуального чтения журналов боевых действий сторонними наблюдателями, в программном комплексе реализован собственный алгоритм зашифровки данных.

Данная подсистема интегрирована в каждый боевой модуль (РЛС, ЗРДН, СПРО) в виде функции write_log и работает по следующему алгоритму:

Перехват сообщения: при возникновении любого значимого события, система формирует строку отчета в формате [ДД.ММ ЧЧ:ММ:СС:МСС] [ИМЯ_СИСТЕМЫ] [СООБЩЕНИЕ].

Унификация кодировки (Base64): сформированная кириллическая строка (в кодировке UTF-8, где один символ занимает несколько байт) передается на вход утилите base64. Флаг -w 0 гарантирует отсутствие переносов строк. Это преобразование переводит сложные байты UTF-8 в безопасный однобайтовый формат ASCII (состоящий из латинских букв и цифр), защищая данные от повреждения на следующем этапе.

Зеркальное отражение: полученная Base64-строка передается по конвейеру встроенной утилите Linux rev, которая выполняет посимвольный реверс. Это действие уничтожает характерные визуальные признаки кодировки Base64, превращая текст в криптографический "шум", не поддающийся расшифровке стандартными автоматизированными сканерами без знания заложенной логики.

Изолированная запись: полученная зашифрованная строка записывается в индивидуальный файл системы с суффиксом _enc.log (например, ZRDN_1_enc.log).

Алгоритм обратного преобразования (Дешифровка):

Дешифровка осуществляется исключительно на стороне Командного пункта (КП) в реальном времени. Скрипт kr.sh считывает зашифрованные файлы построчно и применяет обратный конвейер: rev | base64 -d. Ошибки декодирования (например, при чтении пустых строк) подавляются перенаправлением 2>/dev/null. Восстановленный чистый текст передается для хронологической сортировки и последующего импорта в базу данных SQLite3.

3.5. Информационно-аналитический центр (Алгоритм Командного пункта)

Командный пункт работает в бесконечном цикле:

Мониторинг процессов: КП берет список из 7 PID-файлов. Если файл есть и команда `kill -0 <PID>` возвращает успех — выводит статус зеленым "АКТИВЕН". Иначе — "НЕ АКТИВЕН".

Очистка кэша: КП удаляет старую таблицу в SQLite БД и очищает глобальный текстовый лог `vko_decrypted_all.log`.

Дешифровка данных:

Скрипт ищет все файлы `*_enc.log` в папке.

Читает их построчно. Каждую строку пропускает через дешифратор: `rev | base64 -d`.

Полученный чистый текст записывается в `vko_decrypted_all.log`.

Подготовка SQL-транзакции:

Расшифрованная строка разбивается на переменные с помощью утилиты `awk` (Дата, Время, Имя системы, Сообщение).

Формируются запросы `INSERT INTO logs` и записываются во временный файл `temp_insert.sql`.

Хронологическая сортировка:

Чтобы сообщения от разных систем не шли кусками, скрипт вызывает утилиту `sort`, которая выстраивает глобальный лог `vko_decrypted_all.log` по времени (от старых к новым).

Импорт в БД: Временный SQL-файл выполняется в SQLite (`sqlite3 vko_base.db < temp_insert.sql`), после чего таблица мгновенно заполняется свежими данными.

Сбор статистики (Analytics):

КП делает серию из 5 SQL-запросов `SELECT`.

Подсчитывает COUNT(*) по ключевым словам: "Произведен пуск", "УНИЧТОЖЕНА", "БОЕКОМПЛЕКТ ИСЧЕРПАН", "ОТСУТСТВУЮТ".

Данные выводятся на экран в виде красивых таблиц.

Цикл: Скрипт засыпает на 3 секунды и повторяет всё с пункта 1.

4. Топографическая карта с нанесенными зонами и параметрами работы комплексов ВКО

Карта загружена по ссылке: <https://ibb.co/RGLz5mnt>.

5. Отрывки журналов работы (SQL-запросы к БД и вывод ответов)

Отрывок расшифрованного журнала и пример работы базы данных загружены по следующим ссылкам: <https://ibb.co/KcbCqRMw> и <https://ibb.co/tw6LSpqZ>.

Вывод

В ходе выполнения курсовой работы была успешно спроектирована, разработана и протестирована программная модель автоматизированной системы воздушно-космической обороны (ВКО). Поставленная цель по созданию распределенной многопроцессной архитектуры, способной в режиме реального времени реагировать на динамическую воздушную обстановку, достигнута в полном объеме.

В процессе реализации комплекса были успешно решены следующие инженерные и алгоритмические задачи:

Реализация асинхронной архитектуры: созданы независимые демоны радиолокационной разведки (РЛС) и огневого поражения (ЗРДН, СПРО), взаимодействующие с файловой системой.

Оптимизация боевых алгоритмов: разработан и внедрен математический аппарат для селекции целей по вектору скорости (разделение на баллистические и аэродинамические объекты), а также алгоритм подтверждения поражения по анализу дельты координат.

Обеспечение информационной безопасности: внедрена подсистема защиты журналов боевых действий. Разработанный алгоритм поточного шифрования «Зеркало» (Base64 + строковый реверс).

Агрегация и анализ данных: разработан интерактивный командный пункт с модулем автоматической дешифровки, хронологической сортировки и импорта данных в реляционную базу SQLite3. Настроена бесперебойная выдача статистических сводок о ходе боевых действий (расход боеприпасов, точность систем, критические пропуски целей).

Разработанный программный комплекс продемонстрировал высокую надежность, автономность и эффективность. Результаты тестирования (представленные в журналах работы и SQL-выборках) подтверждают корректность заложенной математической логики и полное соответствие системы заданным тактико-техническим характеристикам варианта.